

MATLAB

4. Grafika a vizualizácia dát



Stredná priemyselná škola elektrotechnická, Karola Adlera 5

Čo nás dnes čaká

1. **Prvý graf** — ako vlastne plot funguje a prečo potrebuje dva vektory
2. **Vzhľad čiary** — farba, štýl, značky, hrúbka; LineSpec aj pomenované parametre
3. **Popis a osi** — xlabel, legend, grid, rozsah osí
4. **Viac kriviek, viac grafov** — hold on, subplot, tiledlayout
5. **Ďalšie typy grafov** — bar, stem, scatter, histogram
6. **Praktické príklady** — harmonické, nabíjanie kondenzátora, dve osi y
7. **Tretí rozmer** — plot3, meshgrid, surf
8. **Uloženie obrázka** — ako graf dostať do protokolu

Všetky obrázky v tejto prednáške sú **skutočné výstupy z MATLABu** — vygenerované skriptom, ktorý nájdete v priečinku `obrazky/`.

1. Prvý graf

Prečo vôbec kresliť

- Doteraz sme výsledky čítali ako čísla v Command Window
- Pri desiatich hodnotách to ide. Pri **tisíc hodnotách** už nie
- Graf ukáže na prvý pohľad to, čo by sme v tabuľke hľadali dlho:
 - kde má priebeh **maximum** alebo **minimum**
 - či hodnoty **rastú**, **klesajú**, alebo **kmitajú**
 - či niektoré meranie **vybočuje** z radu
- V elektrotechnike je graf základná forma dokumentácie — priebeh napätia, charakteristika súčiastky, frekvenčná odozva

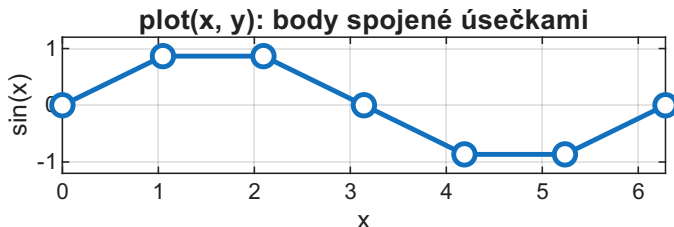
Graf nie je ozdoba. Je to spôsob, ako dáta pochopiť.

plot — ako to naozaj funguje

Funkcia `plot` dostane **dva vektory**: zoznam x -ových a zoznam y -ových súradníc.

```
x = [0 1 2 3 4];  
y = [0 1 4 9 16];  
plot(x, y)
```

- MATLAB si body $(0,0)$, $(1,1)$, $(2,4)$, $(3,9)$, $(4,16)$ vyznačí
- a potom ich **spojí rovnými úsečkami** — nič viac v tom nie je
- **Žiadnu funkciu nepozná**. Nevie, že ide o $y = x^2$. Vidí len body.



Dva vektory – a musia byť rovnako dlhé

```
plot(1:5, 1:6)
```

Error using plot

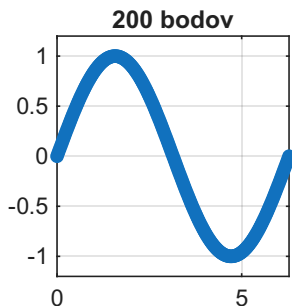
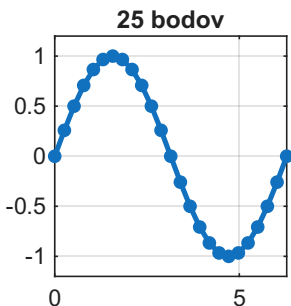
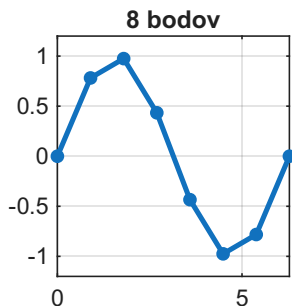
Specify the coordinates as vectors or matrices of the same size, or as a vector and a matrix that share the same length in at least one dimension.

- Päť x -ov a šesť y -ov sa nedá spárovať – MATLAB to odmietne
- **Drobnosť:** na *tvare* (riadok vs. stĺpec) nezáleží. `plot([1 2 3]', [1 2 3])` prejde bez chyby, lebo dĺžka sedí
- Ak zadáme **len jeden** vektor, MATLAB doplní x ako poradie prvkov:

```
plot(y)           % to iste ako plot(1:length(y), y)
```

Kolko bodov je dost'

```
x = linspace(0, 2*pi, 8); % 8 bodov -> hranata ciara  
x = linspace(0, 2*pi, 200); % 200 bodov -> oku hladka krivka  
plot(x, sin(x))
```



- Krivka nie je hladká nikdy – len **dost' jemne nalámaná**, aby to oko nerozoznalo
- Rozumné pravidlo: **100 až 500 bodov** stačí; pri miliónoch sa graf len spomalí

linspace alebo dvojbodka?

Obe vyrobia vektor, ale zadávajú sa inak:

`linspace` — zadám počet bodov

```
x = linspace(0, 1, 5)

x =
    0    0.2500    0.5000    0.7500    1.0000
```

Koncový bod je **vždy presne** zahrnutý.

`:` — zadám krok

```
x = 0:0.25:1

x =
    0    0.2500    0.5000    0.7500    1.0000
```

Koniec sa traťí len vtedy, keď krok vyjde.

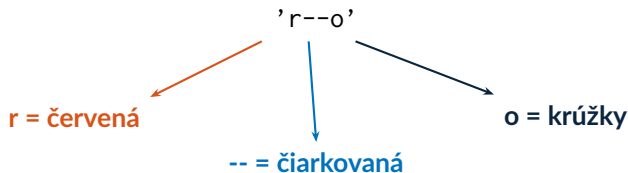
- `0:0.3:1` dá `0 0.3 0.6 0.9` — jednotku už nedosiahne
- Pre graf preto takmer vždy `linspace`: viem, koľko bodov dostanem, a koniec sedí

2. Vzhľad čiary

LineStyle — tri vlastnosti jedným reťazcom

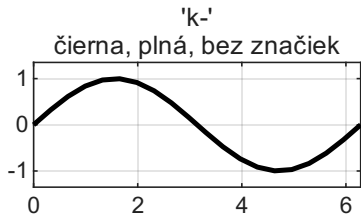
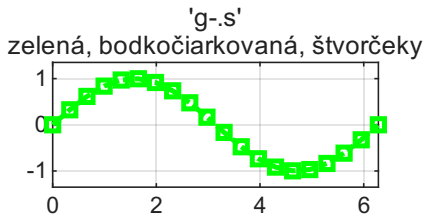
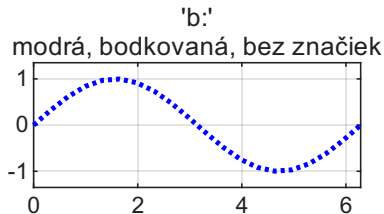
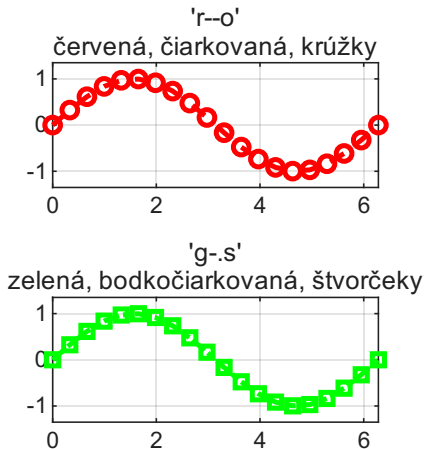
Tretí argument `plot` je krátky reťazec, ktorý naraz určí **farbu**, **štýl** čiary a **značku**:

```
plot(x, y, 'r--o')
```



- Poradie znakov je ľubovoľné — `'r-o'`, `'o-r'` aj `'-or'` znamenajú to isté
- Ktorúkoľvek zložku možno **vynechať**: `'r'` je len farba, `':'` len štýl
- Ak vynecháme štýl aj značku (napr. `'r'`), nakreslí sa **plná čiara**
- Ak zadáme **len značku** (napr. `'o'`), čiara sa **nenakreslí vôbec** — len body

LineSpec v praxi



Farby: písmená, RGB, hex

Osem písmen pre LineSpec

r	červená	m	purpurová
g	zelená	c	azúrová
b	modrá	y	žltá
k	čierna	w	biela

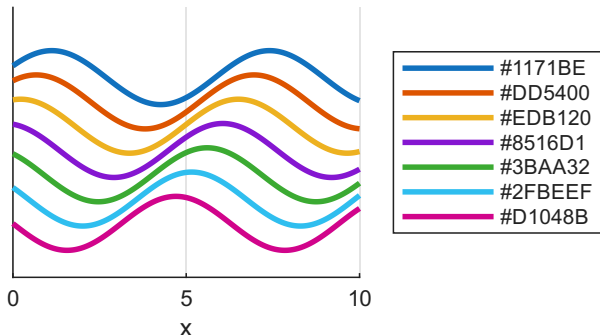
Presná farba cez Color

```
plot(x, y, 'Color', [0.85 0.33 0.10])  
plot(x, y, 'Color', '#D95319')
```

RGB trojica je v rozsahu **0 až 1**, nie 0 až 255.

Hex zápis je reťazec a začína mriežkou.

Keď farbu nezadáme, MATLAB berie zo zoznamu ColorOrder — postupne jednu po druhej, po siedmej sa vráti na začiatok.



Pozor: toto sú hodnoty z **R2026a**. Staršie verzie MATLABu

Štýly čiar a značky

Štýly čiar



Značky (markery)



- Značka sa kreslí v **každom bode** vektora — pri 500 bodoch je graf nečitateľný
- Riešenie: buď kresliť menej bodov, alebo MarkerIndices:

```
plot(x, y, '-o', 'MarkerIndices', 1:20:length(x))
```

Pomenované parametre (Name-Value)

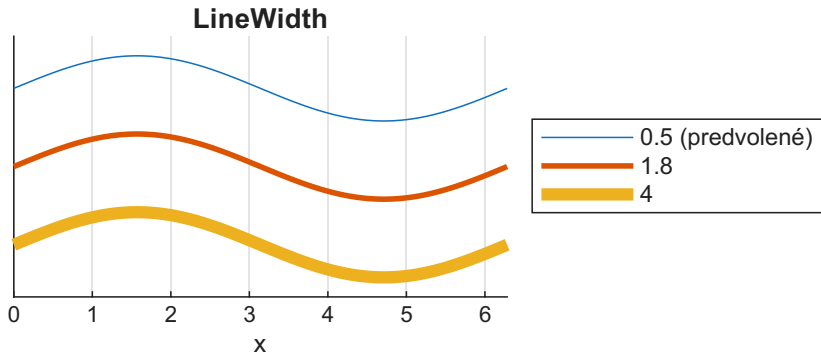
Čo sa nezmesť do LineSpec, zadáme ako dvojicu meno a hodnota:

```
plot(x, y, 'o-', ...  
      'LineWidth',    2, ...  
      'Color',        [0 0.45 0.74], ...  
      'MarkerSize',   8, ...  
      'MarkerFaceColor', 'w', ...  
      'MarkerEdgeColor', 'k')
```

Tri bodky ... na konci riadku znamenajú „príkaz pokračuje na ďalšom riadku“.

LineWidth	hrúbka čiary v bodoch; predvolene 0.5
Color	farba čiary (RGB trojica, hex alebo písmeno)
LineStyle	'-', '--', ':', '-.' alebo 'none'
Marker	značka, napr. 'o'; 'none' ju vypne
MarkerSize	veľkosť značky; predvolene 6
MarkerFaceColor	výplň značky; predvolene 'none' (prázdna)
MarkerEdgeColor	obrys značky
DisplayName	text pre legendu

Hrúbka čiary – vidieť rozdiel

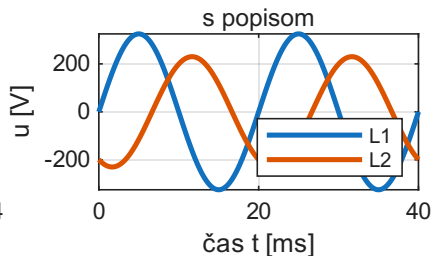
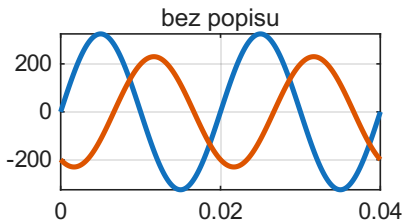


- Predvolených 0.5 je na premietanie a na tlač **príliš tenkých**
- Na slajdy a do protokolu používajte 'LineWidth', 1.5 a viac

3. Popis a osi

Graf bez popisu je nepoužitelný

```
plot(t, u1); hold on; plot(t, u2); hold off  
xlabel('čas t [ms]'); ylabel('napatie u [V]')  
title('Priebeh dvoch faz siete 50 Hz')  
legend('faza L1', 'faza L2', 'Location', 'southeast')  
grid on
```



Popisné funkcie – prehľad

<code>xlabel('text')</code>	popis vodorovnej osi
<code>ylabel('text')</code>	popis zvislej osi
<code>title('text')</code>	nadpis nad grafom
<code>subtitle('text')</code>	menší podnadpis pod nadpisom
<code>legend(...)</code>	vysvetlivky ku krivkám
<code>grid on / grid off</code>	mriežka; <code>grid minor</code> pridá jemnú
<code>text(x, y, 'text')</code>	text na konkrétne miesto v grafe
<code>xline(5) / yline(0)</code>	zvislá / vodorovná čiara cez celý graf

■ Všetky berú aj Name-Value dvojice: `xlabel('t [s]', 'FontSize', 14)`

■ `xline` a `ylines` vedia aj vlastný popisok:

```
ylines(12, '--', 'U = 12 V')
```

V popisoch funguje **TeX**: `'uC [V]'` dá dolný index, `'x2'` horný, `'\tau'` grécke písmeno. Ak to nechceme, pridáme `'Interpreter', 'none'`.

Legenda

Dva spôsoby, ako legende povedať texty:

1. Zoznam v poradí kreslenia

```
plot(x, sin(x))  
hold on  
plot(x, cos(x))  
hold off  
legend('sin(x)', 'cos(x)')
```

Pozor na poradie — ak krivky preusporiadame, legenda bude klamať.

- 'Location' určí umiestnenie: 'north', 'southeast', 'northwest', ...
- 'best' nechá MATLAB vybrať miesto, kde prekryje najmenej čiar
- Prípona outside (napr. 'eastoutside') legendu vysunie mimo osí
- Krivku možno z legendy vynechať: 'HandleVisibility', 'off'

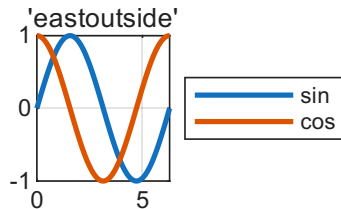
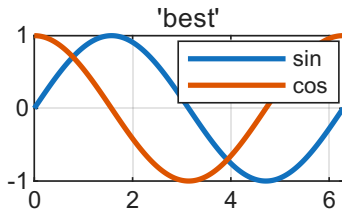
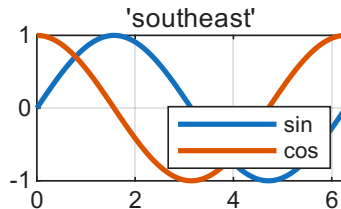
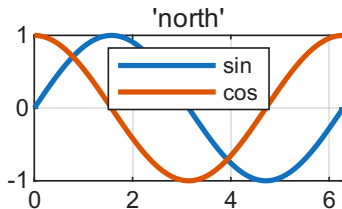
2. DisplayName pri krivke

```
plot(x, sin(x), 'DisplayName', 'sin(x)')  
hold on  
plot(x, cos(x), 'DisplayName', 'cos(x)')  
hold off  
legend
```

Text je priamo pri krivke, takže sa nemôže rozísť.

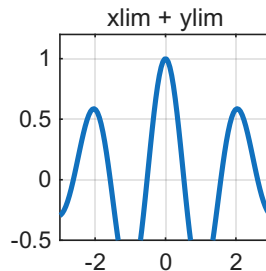
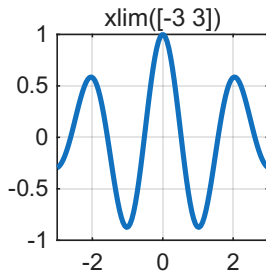
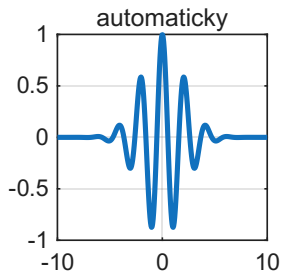
Bezpečnejšie.

Umiestnenie legendy



Rozsah osí

```
xlim([-3 3])           % len vodorovna os  
ylim([-0.5 1.2])       % len zvisla os  
axis([-3 3 -0.5 1.2]) % obe naraz: [xmin xmax ymin ymax]
```



Chyba: xlim([5 1]) — hodnoty musia **rásť**. MATLAB odpovie

Limits must be a 2-element vector of increasing numeric values.

Príkaz axis a jeho prepínače

<code>axis tight</code>	os presne po krajné dáta, bez okrajov
<code>axis equal</code>	rovnaká mierka na oboch osiach (kružnica bude kruhová)
<code>axis square</code>	kreslená plocha bude štvorec
<code>axis padded</code>	necháva malý okraj okolo dát
<code>axis off</code>	skryje osi aj popisy
<code>axis manual</code>	zafixuje rozsah — ďalšie krivky ho už nezmenia
<code>axis auto</code>	vráti automatické prispôsobovanie

- `axis equal` je dôležité vždy, keď má tvar objektu zodpovedať skutočnosti — inak sa kružnica zobrazí ako elipsa
- Ostatné nastavenia osí sa menia cez `gca` (*get current axes*):

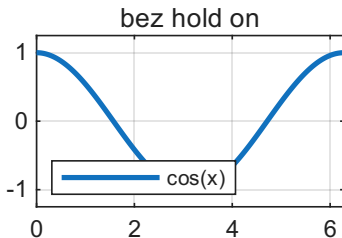
```
set(gca, 'FontSize', 12)
set(gca, 'XTick', 0:pi/2:2*pi)
set(gca, 'YScale', 'log') % logaritmická zvisla os
```

4. Viac kriviek, viac grafov

hold on — najčastejšia chyba v grafike

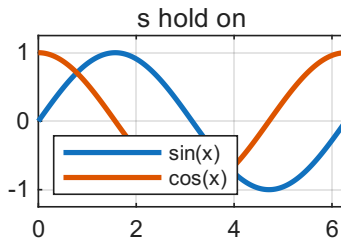
Zle — druhý plot prvý prepíše:

```
plot(x, sin(x))  
plot(x, cos(x))
```



Správne — hold on kreslí navrch:

```
plot(x, sin(x))  
hold on  
plot(x, cos(x))  
hold off
```



- hold off na konci je dobrý zvyk; všimnite si, že po novom plot začína farba **opäť od prvej**

Viac kriviek bez hold

Ak sú krivky známe naraz, stačí ich vymenovať ako dvojice:

```
plot(x, sin(x), 'b-', x, cos(x), 'r--', x, sin(2*x), 'g:')
```

Ak majú spoločnú os x a sú v stĺpcoch matice, MATLAB nakreslí **každý stĺpec** ako samostatnú krivku:

```
Y = [sin(x)', cos(x)', sin(2*x)']; % 3 stĺpce  
plot(x, Y) % 3 krivky, kazda inou farbou
```

- Toto je v MATLABe veľmi častý zápis — napríklad pri meraniach z viacerých kanálov
- **Pozor na orientáciu:** plot berie krivky zo **stĺpcov**, nie z riadkov. Ak máte dáta po riadkoch, transponujte maticou Y'

subplot — mriežka grafov

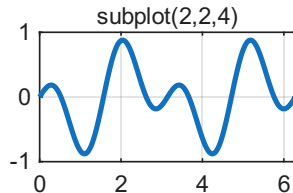
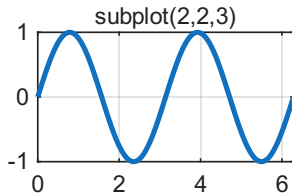
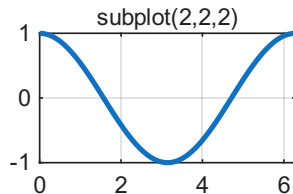
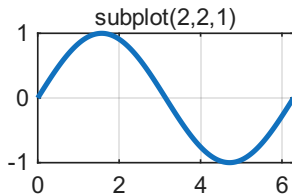
```
subplot(2, 3, 4)
```

Tri čísla znamenajú:

- 2 — počet riadkov
- 3 — počet stĺpcov
- 4 — do ktorého políčka kreslím

Políčka sa číslujú **po riadkoch zľava doprava**:

1	2	3
4	5	6

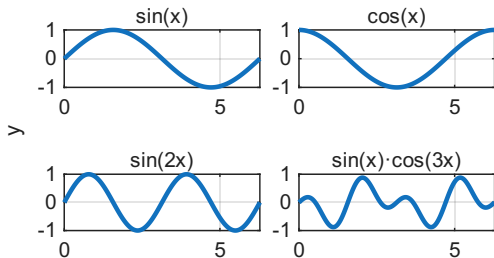


V ukážke vpravo je mriežka 2×2 . **Chyba:** `subplot(2,2,5)` — políčko č. 5 v nej neexistuje, MATLAB odpovie `Index`

tiledlayout — modernejšia náhrada

```
tl = tiledlayout(2, 2, 'TileSpacing', 'compact', 'Padding', 'compact');
nexttile; plot(x, sin(x));
nexttile; plot(x, cos(x));
% ... ďalšie dva nexttile
title(tl, 'Spoločný nadpis')
xlabel(tl, 'x'); ylabel(tl, 'y')
```

spoločný nadpis



- nexttile posúva na ďalšie políčko — nemusíme počítat indexy
- **Spoločný** nadpis a popisy osí pre celú mriežku
- 'TileSpacing', 'Padding': 'loose', 'compact', 'tight'

subplot funguje všade, tiledlayout od R2019b.

Správa okien

figure	otvorí nové prázdne okno
figure(3)	prepne na okno č. 3 (ak nie je, vytvorí ho)
clf	vymaže obsah aktuálneho okna, okno nechá
close	zavrie aktuálne okno
close all	zavrie všetky okná
gcf	<i>get current figure</i> — odkaz na aktuálne okno
gca	<i>get current axes</i> — odkaz na aktuálne osi

- Bez figure kreslí MATLAB stále do **toho istého** okna
- Na začiatok skriptu patrí `close all` — inak sa výsledky miešajú so starými
- Odkaz na okno sa dá uložiť do premennej a použiť neskôr:

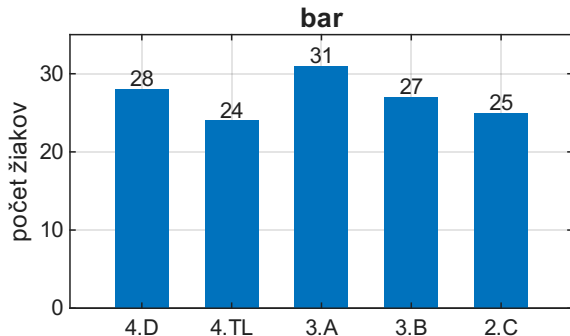
```
f = figure('Name', 'Merania', 'Color', 'w');  
plot(x, y)  
exportgraphics(f, 'graf.png')
```

5. Další typy grafov

bar — porovnanie kategórií

```
trieda = {'4.D', '4.TL', '3.A', '3.B', '2.C'};  
pocety = [28 24 31 27 25];  
bar(categorical(trieda, trieda), pocety, 0.6)  
ylabel('pocet ziaikov')
```

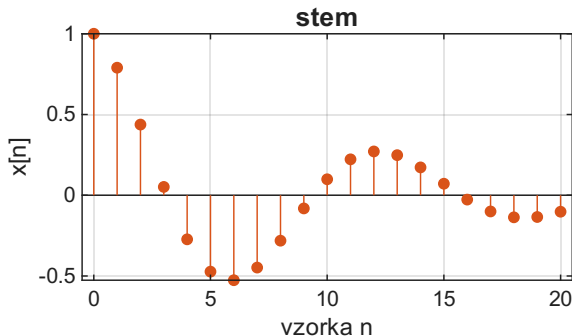
- Tretí argument je **šírka stĺpca** (0 až 1)
- `categorical(trieda, trieda)` zachová **naše poradie** — inak by MATLAB triedy zoradil abecedne
- `bar(Y)` pre maticu dá skupiny; 'stacked' ich naskladá na seba



stem — diskkrétne vzorky

```
n = 0:20;  
stem(n, 0.9.^n .* cos(n/2), 'filled')  
xlabel('poradie vzorky n')  
ylabel('x[n]')
```

- Každá hodnota dostane **vlastnú zvislú čiarku** — žiadne spájanie
- 'filled' vyplní krúžky na vrcholoch
- Toto je **správny graf pre navzorkovaný signál** — medzi vzorkami totiž nič nameraného nie je

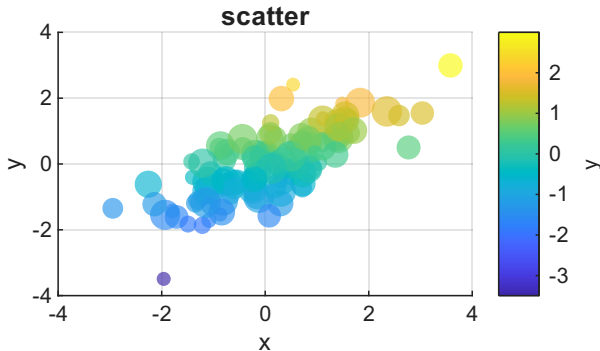


scatter — mrak bodov

```
scatter(x, y, velkosti, farby, 'filled', ...  
        'MarkerFaceAlpha', 0.65)  
colorbar
```

- 3. argument — **veľkosť** každého bodu
- 4. argument — **farba** každého bodu
- MarkerFaceAlpha nastaví priehľadnosť (0 až 1); pri hustom mraku je nevyhnutná
- colorbar pridá farebnú škálu

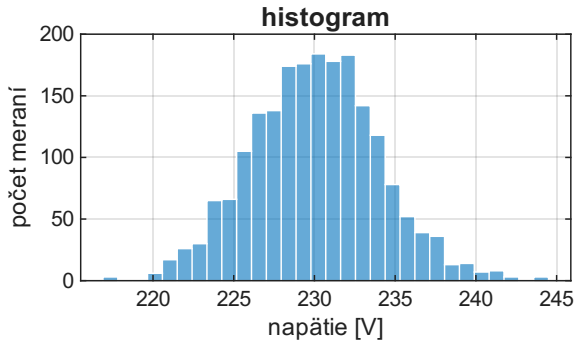
Jeden graf tak unesie až **štyri veličiny** naraz: x , y , veľkosť a farbu.



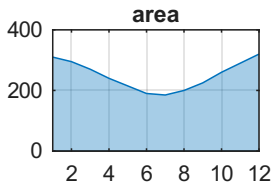
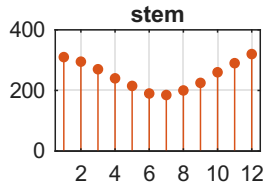
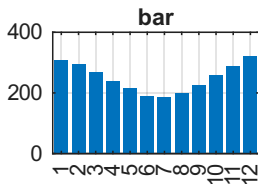
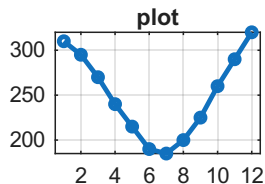
histogram — rozdelenie hodnôt

```
merania = 230 + 4*randn(1, 2000);  
histogram(merania, 30)  
xlabel('namerané napätie [V]')  
ylabel('počet meraní')
```

- Druhý argument je **počet intervalov**; možno zadať aj ich hranice
- 'BinWidth' určí šírku intervalu namiesto počtu
- 'Normalization', 'probability' prepne os y na podiel namiesto počtu



Ktorý graf kedy



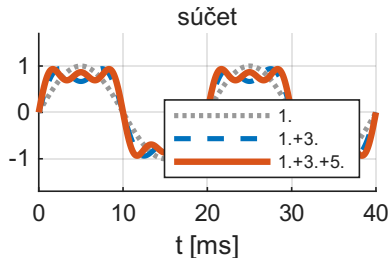
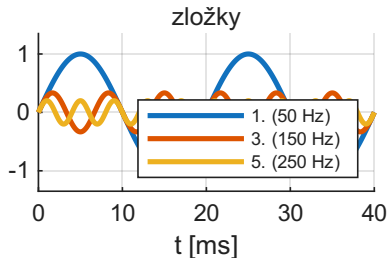
plot	vývoj spojitely veličiny
bar	porovnanie kategórií
stem	diskrétny vzorky
area	objem, naskladané podiely
scatter	vzťah dvoch veličín
histogram	rozdelenie hodnôt
pie	podiely na celku
errorbar	meranie s neistotou
semilog	os y logaritmicky
loglog	obe osi logaritmicky

- **Rovnaké dáta, iný dojem.** Typ grafu nie je vec vkusu — nesie význam
- **plot** pri 12 mesiacoch naznačuje spojitý vývoj. **stem** zdôrazní, že máme len 12 hodnôt

6. Praktické příklady

Superpozícia harmonických

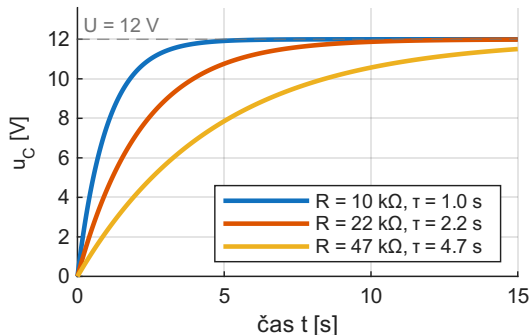
```
t = linspace(0, 0.04, 2000);  
h1 = sin(2*pi*50*t); h3 = sin(2*pi*150*t)/3; h5 = sin(2*pi*250*t)/5;  
plot(t*1000, h1 + h3 + h5, 'LineWidth', 2.2)
```



- Súčet nepárnych harmonických s klesajúcou amplitúdou dáva **obdĺžnikový priebeh**
- Čím viac zložiek pridáme, tým ostrejšie hrany — toto je jadro Fourierovho rozkladu

Nabíjanie kondenzátora

```
U = 12; C = 100e-6;  
R = [10e3 22e3 47e3];  
t = linspace(0, 15, 500);  
  
hold on  
for k = 1:3  
    tau = R(k) * C;  
    plot(t, U*(1 - exp(-t/tau)), ...  
         'DisplayName', sprintf( ...  
             'R = %g kOhm', R(k)/1000))  
end  
yline(U, '--', 'U = 12 V', ...  
      'HandleVisibility', 'off')  
hold off  
legend('Location', 'southeast')
```



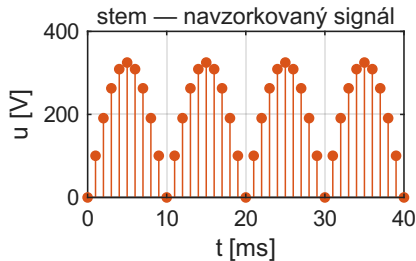
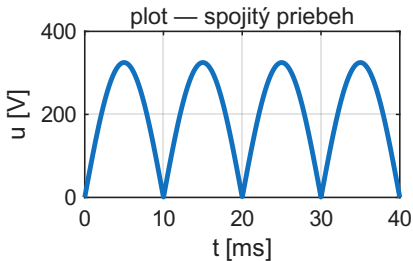
Čo si z toho príkladu odniesť

- **Cyklus a graf idú spolu** — jedna krivka na jeden priechod cyklom
- `hold on` musí byť **pred** cyklom, `hold off` až za ním
- `DisplayName` so `sprintf` vyrobí popisok, ktorý **obsahuje vypočítané čísla** — legenda sa tak nikdy nerozídde s dátami
- `'HandleVisibility', 'off'` udrží pomocnú čiaru `yline` **mimo legendy**
- Prepočet $t \times 1000$ alebo $R/1000$ robíme **až pri kreslení** — vo výpočte ostávajú základné jednotky SI

Toto je presne to miesto, kde sa stretáva všetko z minulej hodiny: **cyklus, formátovaný výpis a funkcie.**

Spojitéy a diskretný pohľad na ten istý signál

```
t = linspace(0, 0.04, 1000); % husto -> spojity dojem
tn = linspace(0, 0.04, 41); % riedko -> vzorky
plot(t*1000, abs(325*sin(2*pi*50*t)))
stem(tn*1000, abs(325*sin(2*pi*50*tn)), 'filled')
```



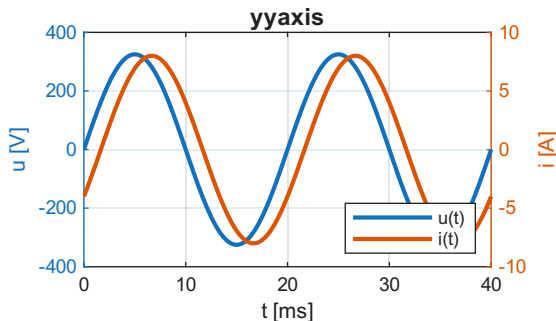
Usmernené napätie. Vľavo ho vidíme tak, ako existuje v obvode; vpravo tak, ako ho „vidí“ mikroprocesor s prevodníkom.

Dve veličiny s rôznymi jednotkami — yyaxis

```
yyaxis left
plot(t*1000, u)
ylabel('napätie u [V]')
ylim([-400 400])

yyaxis right
plot(t*1000, i)
ylabel('prúd i [A]')
ylim([-10 10])
```

- Napätie v stovkách voltov a prúd v jednotkách ampérov sa do jednej osi nezmestia
- yyaxis dá každej veličine **vlastnú zvislú os**
- Z grafu je pekne vidieť **fázový posun** medzi prúdom a napätím



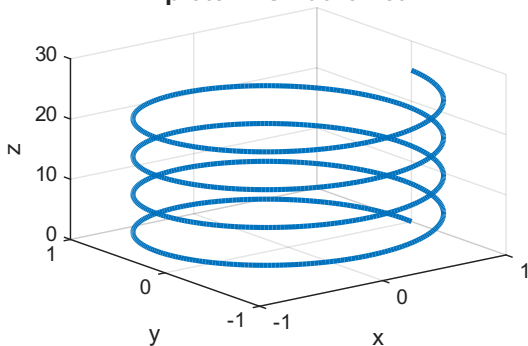
7. Tretí rozmer

plot3 — krivka v priestore

```
t = linspace(0, 8*pi, 1000);  
plot3(cos(t), sin(t), t, 'LineWidth', 2)  
xlabel('x'); ylabel('y'); zlabel('z')  
grid on  
view(-37.5, 25)
```

- plot3 potrebuje **tri** vektory rovnakej dĺžky
- zlabel popíše tretiu os
- view(azimut, elevácia) nastaví uhol pohľadu v stupňoch
- V otvorenom okne sa dá grafom otáčať myšou

plot3 — skrutkovica



meshgrid — mriežka pre plochy

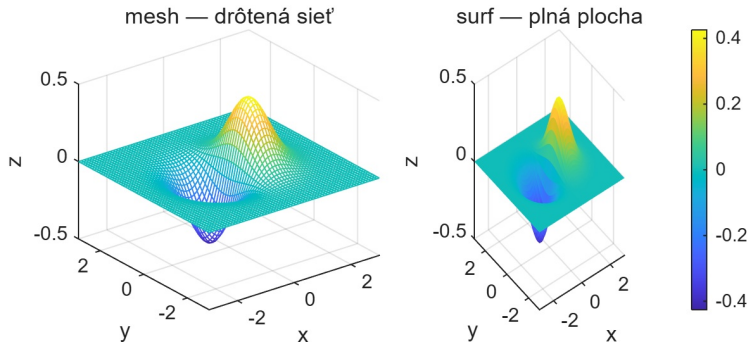
Na plochu nestačia vektory. Potrebujeme **maticu** hodnôt — pre každý bod mriežky jednu:

```
[X, Y] = meshgrid(linspace(-3, 3, 60));  
Z = X .* exp(-X.^2 - Y.^2);  
surf(X, Y, Z, 'EdgeColor', 'none')
```

- meshgrid z jedného vektora vyrobí dve matice: x má v každom riadku to isté, y v každom stĺpci
- Vďaka tomu dvojica (X(i,j), Y(i,j)) prejde **všetky body mriežky**
- Vo vzorci pre z musia byť **bodové operácie** .*, .^, ./ — počítame prvok po prvku, nie maticovo

mesh	drôtená sieť, plochy priesvitné
surf	plná farebná plocha
contour	vrstevnice, pohľad zhora
'EdgeColor', 'none'	skryje čiary siete — pri hustej mriežke nutné

mesh a surf



Farba zodpovedá výške z . Škálu mení `colormap` — napr. `colormap(parula)`, `colormap(jet)`, `colormap(gray)`; `colorbar` ju zobrazí vedľa grafu.

8. Uloženie obrázka

Ako dostať graf do protokolu

```
exportgraphics(gca, 'graf.png', 'Resolution', 300)  
exportgraphics(gcf, 'graf.pdf', 'ContentType', 'vector')
```

exportgraphics 'Resolution' 'ContentType', 'vector' 'BackgroundColor'	odporúčaný spôsob (od R2020a); orezáva biely okraj body na palec pre rastrový formát; do tlače aspoň 300 PDF/EPS, ktoré sa dá ľubovoľne zväčšovať napr. 'white' alebo 'none' (priehľadné)
saveas(gcf, 'graf.png') print('-dpng', '-r300', 'graf')	starší príkaz, necháva veľký biely okraj najstarší, najviac nastavení

- gca uloží **len osi**, gcf **celé okno** aj s viacerými subplot-mi
- Do dokumentu, ktorý sa bude tlačiť, dávajte **vektorové PDF** — nikdy nebude rozmazané
- Do prezentácie alebo na web stačí PNG s rozlíšením 150 až 300

9. Zhrnutie

Časté chyby

- **Zabudnutý `hold on`** — v grafe ostane len posledná krivka
- **Rôzne dlhé vektory** — Specify the coordinates as vectors `...` of the same size
- **Málo bodov** — krivka je hranatá; pomôže `linspace` s väčším počtom
- **Značka v každom bode** — pri 500 bodoch sa graf zlepí; použite `MarkerIndices`
- **Legenda v inom poradí** než krivky — istejšie je `DisplayName`
- **Chýbajúci `xlabel` a `ylabel`** — graf bez jednotiek nič nedokazuje
- `xlim([5 1])` — hranice musia rásť
- `subplot(2,2,5)` — políčko mimo mriežky
- **Zabudnuté `close all`** na začiatku skriptu — nové dáta v starom okne
- **Maticové `*` namiesto `.*`** vo vzorci pre `z` — plocha nevyjde

Zhrnutie

- `plot(x, y)` len **spojí body úsečkami** — o funkcii nič nevie
- Vektory musia byť **rovnako dlhé**; hustotu bodov volíme cez `linspace`
- Vzhľad: krátky **LineSpec** (`'r-o'`) alebo presné **Name-Value** dvojice
- `xlabel`, `ylabel`, `title`, `legend`, `grid on` — **bez nich graf nie je hotový**
- `hold on` pre viac kriviek, `subplot` alebo `tiledlayout` pre viac grafov
- Typ grafu nesie význam: `plot` spojité, `stem` vzorky, `bar` kategórie, `histogram` rozdelenie
- Ukladáme cez `exportgraphics` — do tlače vektorové PDF

Nabudúce: práca so súbormi — načítame namerané dáta zo súboru a hneď ich vykreslíme.

Ďakujem za pozornosť

Otázky a diskusia
